The Transformer Codebook: A Line-by-Line Guide

Welcome to your complete textbook on the Transformer! We are going to look at the exact Python code from your notebook, read it line-by-line, and understand the mathematics happening behind the scenes. We will use the example sentence: **"anugrah is learning"**.

Chapter 1: The Setup & Tokenization

Before a computer can process words, it must turn them into numbers using a dictionary (vocabulary).

```
# 1. Define the Vocabulary
new_vocab = {'<pad>': 0, '<unk>': 1, '<cls>': 2, '[sep]': 3, 'this': 4, 'is':
5, ... 'learning': 9, ... 'anugrah': 38}

def prepare_transformer_inputs(input_text, vocab, max_seq_len):
    # Step 1: Add special tokens (<cls> at start, [sep] at end)
    # Step 2: Convert to IDs
    # Step 3: Pad to max_seq_len (10)
    # Step 4: Create attention mask
    return input_ids, attention_mask, token_type_ids
```

Line-by-Line Breakdown:

- `new_vocab`: This is our dictionary. Every word has a unique Roll Number.
- `prepare_transformer_inputs()`: This function takes "anugrah is learning". It adds a start tag and an end tag. If the sentence is shorter than 10 words, it adds 0s (padding) to make the length exactly 10.
- **Matrix Calculation (Output):** For "anugrah is learning", the output is a 1D tensor (array):

  Input IDs = [1, 38, 5, 9, 1, 0, 0, 0, 0, 0]

  (Shape: 1 batch × 10 sequence length)

Chapter 2: Word & Positional Embeddings

Numbers alone don't capture meaning. We convert each Roll Number into an 8-dimensional vector (like giving marks in 8 subjects).

```
d_model = 8
embedding = nn.Embedding(num_embeddings=vocab_size, embedding_dim=d_model)
embedded_input = embedding(input_ids) # Shape becomes [1, 10, 8]

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_seq_len=5000):
        # ... setup sine and cosine waves ...
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

    def forward(self, x):
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)
```

Line-by-Line Breakdown & Math:

- `nn.Embedding`: This creates a lookup table. For the word "anugrah" (ID 38), it pulls out 8 decimal numbers. The matrix expands from *1x10* to *1x10x8*.
- `PositionalEncoding`: Transformers don't read left-to-right; they read all words at once. We calculate Sine and Cosine waves based on the word's position (0 to 9) and add it to the embedding vector.
- **Vector Addition:** *Word Vector [0.12, -1.5...] + Position Vector [0.00, 1.87...] = Final Input [0.12, 0.28...]*

Chapter 3: The Encoder (Multi-Head Attention)

This is where words look at each other to understand context.

```
class TransformerEncoderLayer(nn.Module):
    def forward(self, x, mask=None):
        Q = self.Wq(x) # Query Matrix
        K = self.Wk(x) # Key Matrix
        V = self.Wv(x) # Value Matrix

        # Split into multiple heads
        Q = Q.view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
        # ... same for K and V ...

        attention_output = self._calculate_attention(Q, K, V, mask=mask)
        # ... Add & Norm, Feed Forward ...
```

The Matrix Mathematics here:

- `Q = self.Wq(x)`: We multiply our input ***X (10x8)*** by a weight matrix ***W (8x8)*** to get the Query Matrix ***Q (10x8)***. We do the same for Key (***K***) and Value (***V***).
- **The Split:** Our code has `num_heads = 2`. It takes the 8 dimensions and splits them into 2 groups of 4. So now we have two separate ***10x4*** matrices thinking independently!
- **Attention Equation:** Inside `_calculate_attention`, it does:
  Attention Scores = Softmax((Q * K^T) / √4)
  Q (10x4) * K^T (4x10) results in a ***10x10*** matrix. This matrix tells us how much attention word #3 ("is") gives to word #4 ("learning").
- We multiply these percentages by ***V*** to get the final context vector.

[Input 10x8] → Split into 2 Heads [10x4, 10x4] → Q*K^T Attention Math → Combine back to [10x8]

Chapter 4: The Decoder & The Encoder-Decoder Bridge

The Decoder generates the translated sentence. The most important part is how the **Encoder's output passes into the Decoder**.

```
class TransformerDecoderLayer(nn.Module):
    def forward(self, x, enc_output, src_mask, tgt_mask):
        # 1. Masked Self-Attention (Decoder looks at itself)
        # ... (calculates Q_masked, K_masked, V_masked) ...
        x = self.layer_norm1(x + mha_output)

        # 2. Multi-Head Encoder-Decoder Attention (THE BRIDGE)
        Q_enc_dec = self.enc_dec_Wq(x)          # Q comes from Decoder
        K_enc_dec = self.enc_dec_Wk(enc_output) # K comes from Encoder
        V_enc_dec = self.enc_dec_Wv(enc_output) # V comes from Encoder

        enc_dec_attn_output = self._calculate_attention(Q_enc_dec, K_enc_dec,
V_enc_dec, mask=src_mask)
        x = self.layer_norm2(x + enc_dec_attn_output)
```

The Vector Addition & Passing of Output:

This is the exact interaction you asked about! Let's say we are translating to Hindi. The Decoder is trying to guess the next Hindi word.

- `enc_output`: This is the *10x8* matrix we generated from the Encoder in Chapter 3. It contains the full meaning of "anugrah is learning".
- `x`: This is the Decoder's current state. It generates the Query Q (e.g., "I need a Hindi verb that matches the English action").
- `K and V`: These are generated entirely from the `enc_output`.
- **The Calculation:** $Q\_decoder * K\_encoder^T$. The Decoder matrix matches itself against the Encoder matrix to find out which English word to pay attention to right now. If it's translating "learning", the math will output a high percentage score (like 0.95) connecting to the 4th English word vector!
- **Add & Norm:** The result is then added back to the Decoder's vector: $x = x + enc\_dec\_attn\_output$.

Chapter 5: The Final Output (Linear Layer)

```
class FullTransformer(nn.Module):
    def forward(self, src, tgt, src_mask, tgt_mask):
        enc_output = self.encoder(src, src_mask)
        dec_output = self.decoder(tgt, enc_output, src_mask, tgt_mask)

        # Final linear layer
        output = self.output_linear(dec_output)
        return output
```

The Final Calculation:

- The Decoder outputs a *10x8* matrix. But we need to predict a word out of 39 possible words in our dictionary.
- `self.output_linear` acts as a multiplier. It multiplies our *10x8* matrix by a weight matrix of *8x39*.
- **Final Matrix Shape:** *10x39*. For each of the 10 positions, we now have 39 numbers. The highest number among those 39 indicates the predicted word!

[Decoder Matrix 10x8] × [Linear Weights 8x39] = [Final Prediction Matrix 10x39]
Highest score = Next Word Predicted!